

Fintech Multi-Agent Orchestrator

A Technical Guide

Contents

1	What Is This System?	3
1.1	The analogy	3
1.2	Key vocabulary	3
2	Technology Choices	4
2.1	Why Python?	4
2.2	Why async throughout?	4
3	Architecture	4
4	How a Request Flows	5
5	The Orchestration Graph	6
5.1	What LangGraph does	6
5.2	The graph structure	6
5.3	The task state	7
6	The Agent Layer	7
6.1	Overview of all agents	7
6.2	The Supervisor: routing	8
6.3	The DataAgent: tool calling	8
6.4	Structured output agents	8
6.5	How agents share data	9
6.6	The Validator	9
7	The Tools Layer	10
7.1	sql_query	10
7.2	excel_query	10
7.3	document_search	10
8	The Gateway Layer	11
8.1	Async submission	11
8.2	Input sanitization	11
8.3	Authentication and roles	11

8.4	Rate limiting	11
8.5	Graceful shutdown	12
9	State Persistence with Redis	12
9.1	Why Redis instead of a SQL database?	12
9.2	What is stored	12
9.3	Serialisation	12
9.4	The audit trail	13
10	Cost Tracking	13
11	Engineering Decisions	14
11.1	Supervisor pattern over dynamic planning	14
11.2	One validator with one retry	14
11.3	Structured output where decisions are made	14
11.4	SQLite placeholder over a real database	14
11.5	Three-tier sanitization	14
11.6	Paths anchored to the project root	15
12	File Reference	15
12.1	src/orchestrator/	15
12.2	src/agents/	15
12.3	src/tools/	15
12.4	src/gateway/	16
12.5	src/memory/ and src/audit/	16
12.6	src/utlis/	16
12.7	scripts/ and data/	16
13	What Is Not Production-Ready	17
14	Where to Start Reading the Code	17

1 What Is This System?

This is a question-answering system for a finance operations team. An analyst types a question like:

“Which funds have breached their risk limits today, and what does the mandate say about the cash minimum?”

The system figures out what the question is asking for, fetches the relevant data, runs the appropriate analysis, checks its own answer for quality, and returns a formatted report.

The key idea is **orchestration**. A single AI model call cannot reliably do all of this. Instead, the system breaks the work into steps and runs a chain of specialised components – each responsible for one part – passing results forward until the job is done.

1.1 The analogy

Think of a small internal consulting team. A client brief arrives. A project manager reads it and assigns it. A data analyst pulls the numbers. A risk analyst checks them against compliance rules. A report writer assembles the findings. A reviewer approves the draft before it goes out.

This system automates exactly that workflow. Each role is replaced by a software component backed by an AI model.

1.2 Key vocabulary

The following terms are used throughout this document.

Term	Meaning
LLM	Large Language Model. The AI that generates text – Claude, GPT-4, etc. Think of it as a function: text goes in, text comes out.
Agent	A module that uses an LLM for one specific job. Each agent has a system prompt that defines its role.
Tool	A regular Python function the LLM is allowed to call. The LLM requests “run this SQL query”; the tool executes it and returns the result.
Structured output	Forcing the LLM to return a specific JSON shape instead of free text. The shape is validated against a Pydantic model.
State machine	A program that moves through a fixed set of phases (supervising, executing, validating, ...) via defined transitions.

Table 1: Vocabulary used in this document

2 Technology Choices

Library	Role	Key reason for choosing it
LangGraph	Orchestrates the workflow	Built specifically for multi-step LLM pipelines
LangChain	LLM and tool interface	Abstracts provider differences (Claude vs GPT)
FastAPI	HTTP API	Async-native; generates docs automatically
Pydantic	Data modelling and validation	Catches bad data at runtime with clear errors
Redis	Task state and audit storage	Fast, async, supports TTL and sorted sets
SQLite	Placeholder database	Zero setup; swap for a real DWH in production
ChromaDB	Document search	Runs in-process; no extra service needed
Claude	LLM provider	Strong structured output; 200k token context
slowapi	Rate limiting	One decorator per route
structlog	Logging	Outputs structured JSON

Table 2: Full technology stack

2.1 Why Python?

The LangChain and LangGraph ecosystems are Python-first. The entire industry converged on Python for LLM work. Using another language would mean reimplementing these libraries from scratch.

2.2 Why async throughout?

LLM API calls take 2–20 seconds. A blocking server could handle only one request at a time. Python’s `asyncio` lets a single process handle many in-flight requests concurrently. FastAPI, LangGraph, Redis, and the database clients are all async-native.

3 Architecture

The system has four layers, each with a single clear responsibility.

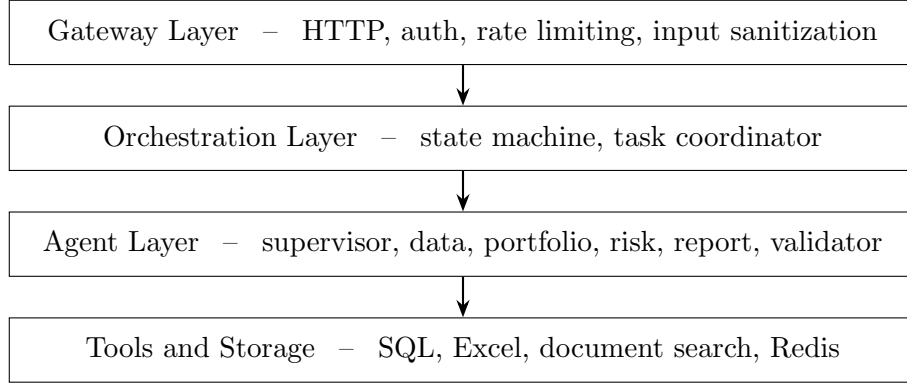


Figure 1: Four-layer architecture. Dependencies only flow downward.

The important constraint is that dependencies only flow downward. The agent layer imports from the tools layer, but the tools layer never imports from agents. This means each layer can be tested in isolation.

4 How a Request Flows

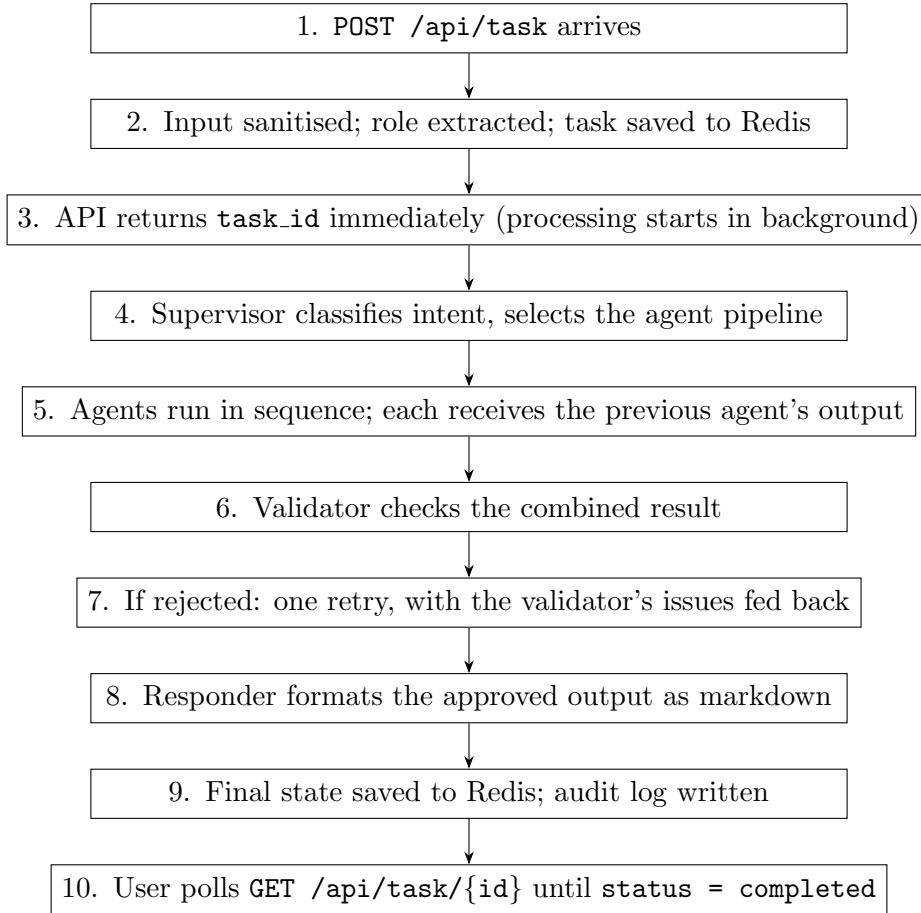


Figure 2: End-to-end request lifecycle

Step 3 is the key design point: the API returns immediately. The actual LLM processing happens in a background async task. This means the HTTP connection does not stay open while the model works.

The user polls step 10 until the status changes. This is called the **polling pattern** and is common for background jobs that take more than a few seconds.

5 The Orchestration Graph

5.1 What LangGraph does

LangGraph is a library for writing workflows as directed graphs. Each **node** is an async Python function. Each **edge** says which node runs next. Some edges are **conditional** – the next node depends on a value in the current state.

The entire state of a task lives in a single Python dictionary called `OrchestratorState`. Every node receives this dictionary and returns an updated copy. Nothing is stored globally.

5.2 The graph structure

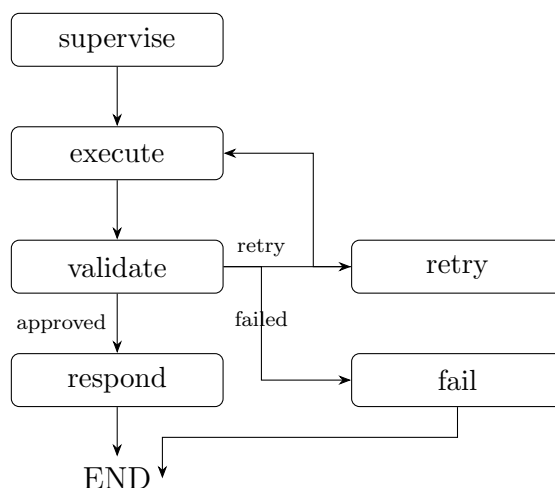


Figure 3: Graph nodes and transitions. The retry node loops back into execute.

Node	What it does
supervise	Classifies the user’s intent and selects the list of agents to run.
execute	Runs each agent in sequence, passing each agent’s output to the next.
validate	Checks the combined output for completeness, accuracy, and safety.
retry	Increments the retry counter and routes back to execute. Allowed once.
respond	Formats the approved output into a final markdown response.
fail	Handles the case where validation keeps rejecting or the budget is exceeded.

Table 3: What each graph node does

5.3 The task state

Every node reads from and writes to the same object. The fields below are the most important ones.

Listing 1: Core fields of OrchestratorState

```
1 class OrchestratorState(TypedDict):
2     task_id: str
3     user_id: str
4     task: str          # the user's original question
5
6     intent: str        # e.g. "risk_report" -- set by supervise
7     agents_selected: list # e.g. ["data", "risk", "report"]
8     agent_results: dict  # agent_name -> {status, result}
9
10    validation_result: ValidationResult
11    retry_count: int
12
13    final_response: str   # the markdown answer returned to the user
14    cost_tracking: CostTracking
```

Because all state is in one place, debugging is straightforward: print the state at any node and you can see exactly where things stand.

6 The Agent Layer

6.1 Overview of all agents

Agent	LLM pattern	What it does
Supervisor	Structured output	Reads the task; decides which agents to call
DataAgent	Tool calling loop	Fetches data via SQL, Excel, and document queries
PortfolioAgent	Structured output	Analyses positions, NAV, P&L, attribution
RiskAgent	Structured output	Detects limit breaches and exposure flags
ReportAgent	Free text	Composes a markdown report from all prior outputs
Validator	Structured output	Quality check: approves or rejects the combined output
Responder	Free text	Final formatting pass before the user sees the result

Table 4: All agents and their LLM usage patterns

Notice the pattern: agents that make **decisions** (Supervisor, Validator) or produce **structured data** (Portfolio, Risk) use structured output. Agents that write **prose** (Report, Responder) use free text – constraining markdown to a JSON schema would add no benefit.

6.2 The Supervisor: routing

The Supervisor’s job is classification. It takes the user’s question and selects the right agent pipeline.

Intent	Agent pipeline
data_query	data
portfolio_query	data → portfolio
risk_compliance	data → risk
risk_report	data → risk → report
full_analysis	data → portfolio → risk → report
unknown	data (safe fallback)

Table 5: Intent categories and their agent pipelines

After the LLM responds, a **sanitize step** checks whether the returned intent and agent names are valid. If the LLM hallucinated an agent name like "**compliance_checker**", the sanitizer replaces the pipeline with the canonical one for that intent. This stops bad routing from reaching the execution step.

6.3 The DataAgent: tool calling

The DataAgent is the only agent that calls external tools. The mechanism is a loop:

1. Send the task to the LLM along with a list of available tools.
2. The LLM responds by requesting one or more tool calls (e.g. “run this SQL query”).
3. The code executes each call and feeds the result back to the LLM as a message.
4. If the LLM has enough data, it stops calling tools and returns a plain text summary.
5. If not, it makes more tool calls. The loop runs up to five iterations.

This means the LLM decides **what to query and when to stop**, rather than the code hardcoding a fixed query per question type.

6.4 Structured output agents

Portfolio, Risk, and Validator agents use `.with_structured_output()`, which forces the LLM to return a JSON object matching a specific Python class.

Listing 2: RiskAgent output schema (simplified)

```
1 class LimitBreach(BaseModel):
2     rule_id: str
3     fund_id: str
4     rule_name: str
5     limit_value: float
6     current_value: float
```



```

7
8 class RiskResult(BaseModel):
9     overall_status: str          # "ok" / "warning" / "breach"
10    flags: list[RiskFlag]         # every rule evaluated
11    breaches: list[LimitBreach]  # only the violated ones
12    summary: str
13
14 self.llm = llm.with_structured_output(RiskResult)

```

If the LLM forgets a field or uses the wrong type, Pydantic raises an exception immediately. Compare this to parsing free text, where a different phrasing silently produces wrong data.

6.5 How agents share data

Each agent receives `previous_results` – a dictionary containing every earlier agent’s output. The ReportAgent, which runs last, collects all prior outputs and passes them as context to the LLM.

Listing 3: ReportAgent assembling context from prior agents

```

1 for agent_name in ["data", "portfolio", "risk"]:
2     entry = previous_results.get(agent_name, {})
3     result = entry.get("result", "no output")
4     # Pydantic models are serialised to JSON so the LLM can read them
5     if hasattr(result, "model_dump"):
6         result = json.dumps(result.model_dump(), indent=2)
7     # ... append to context block

```

The ReportAgent does not re-query the database. It works entirely from what was already fetched.

6.6 The Validator

The Validator reads all agent outputs and decides whether the result is good enough to return to the user. Its checks are:

1. **Completeness** – does the output address what was asked?
2. **Consistency** – do figures match across agents?
3. **Safety** – no personally identifiable information in the output.
4. **Hallucination** – every number cited must appear in the source data.
5. **Breach surfaced** – if `limit_rules` shows `breached=1`, the output must say so.

If the Validator sets `approved = False`, the graph routes to the retry node. The `issues` list is passed back to every agent as `validation_feedback` so they know what to fix. One retry is allowed; a second rejection causes the task to fail.

7 The Tools Layer

Tools are ordinary Python functions – no LLM involved. They perform the actual I/O.

7.1 `sql_query`

Runs SELECT queries against a SQLite database. Two safety checks are applied before any query reaches the database:

1. The query must begin with **SELECT**. Anything else (INSERT, DROP, UPDATE) is rejected immediately.
2. A pattern check blocks dangerous constructs: semicolons (used in multi-statement injection), **UNION SELECT**, SQL comment sequences (`--`), and DDL keywords.

The database is seeded in memory at startup with fake data: three funds, nine positions, five trades, seven NAV records, and five limit rules – two of which are deliberately breached. To connect to a real data warehouse, change one function: `get_db()` in `sql.py`.

7.2 `excel_query`

Reads `.xlsx` and `.csv` files from the `data/` directory. Paths are resolved relative to the project root (via `__file__`), not the current working directory, so they work correctly regardless of where the server is started. Directory traversal is blocked – a path like `../../etc/passwd` is rejected before the file is opened.

7.3 `document_search`

This tool lets the LLM answer questions about documents it was never trained on – fund mandates, policy documents, prospectuses. The technique is called **Retrieval-Augmented Generation (RAG)**.

Step 1 – Ingest (run once). Running `python scripts/ingest_docs.py` reads each document, splits it into 500-character passages, converts each passage into a vector embedding, and stores these in ChromaDB.

A vector embedding is a list of numbers (384 of them) that encodes the meaning of a piece of text. Passages with similar meaning produce similar numbers. This enables meaning-based search rather than keyword matching.

Step 2 – Search (on every query). When the LLM calls `document_search("cash minimum requirement")`, ChromaDB converts the query into an embedding and finds the stored passages whose embeddings are most similar. Those passages are returned to the LLM as readable text.

The LLM can then quote and cite documents that were never in its training data.

8 The Gateway Layer

8.1 Async submission

Tasks are not processed during the HTTP request. The API saves the task to Redis, starts a background async task, and returns a `task_id` within milliseconds. The user then polls the status endpoint until processing is complete.

8.2 Input sanitization

Three sanitization methods exist because the appropriate level of strictness depends on context.

Method	Used for	Blocks
<code>check_for_injections()</code>	Natural language task text	Shell metacharacters only
<code>validate_sql_query()</code>	SQL queries before execution	UNION, semicolons, DDL
<code>validate_sql_param()</code>	Raw parameter values	All SQL keywords

Table 6: Three-tier sanitization and when each applies

The natural language method deliberately does not block SQL keywords. An ops user asking “select the top 10 funds by AUM” or “create a risk report” uses those words naturally. Blocking them would make the system unusable.

8.3 Authentication and roles

Users authenticate with a JWT (JSON Web Token) – a signed string that encodes the user’s identity and role. The server verifies the signature using a secret key. No session state is stored on the server.

Role	Tables accessible
<code>ops_read</code>	funds, positions, nav_history, trades
<code>ops_write</code>	all of the above, plus limit_rules
<code>risk_read</code>	funds, positions, limit_rules, nav_history
<code>admin</code>	all tables

Table 7: Role-to-table access matrix

The role is injected into the task context at submission time. The DataAgent reads it and includes a system message telling the LLM which tables it is allowed to query.

8.4 Rate limiting

Implemented with `slowapi` via a per-route decorator. Task submission is limited to 20 requests per minute per IP (each task triggers multiple LLM calls). Status reads

are limited to 100 per minute. The auth endpoint is limited to 10 per minute.

8.5 Graceful shutdown

When the server receives a SIGTERM signal (e.g. `docker stop`), it waits up to 30 seconds for any running tasks to finish, cancels tasks still running after the timeout, and closes the Redis connection. This is handled by FastAPI's `lifespan` context manager.

9 State Persistence with Redis

9.1 Why Redis instead of a SQL database?

Tasks need automatic expiry – Redis TTL handles this natively. A SQL database would need a scheduled cleanup job. The audit log is an append-only time-ordered sequence – Redis sorted sets (score = Unix timestamp) are the right data structure for this. All reads and writes are simple key lookups with no joins or transactions. Redis is faster and simpler for this access pattern.

9.2 What is stored

Key	Type	Expires	Contains
<code>task:{id}</code>	String	24 hours	Full serialised <code>OrchestratorState</code>
<code>session:{id}:tasks</code>	Set	24 hours	Task IDs for this session
<code>session:{id}:history</code>	Sorted set	7 days	Task summaries, sorted by time
<code>audit:{session_id}</code>	Sorted set	90 days	Audit log entries, sorted by time

Table 8: Redis key structure and retention

9.3 Serialisation

`OrchestratorState` contains Pydantic models and Python enums, neither of which is directly JSON-serialisable. Before storing, the serialiser converts them:

Listing 4: Converting special types before storing in Redis

```
1 for key, value in state.items():
2     if hasattr(value, "model_dump"):      # Pydantic model
3         serializable[key] = value.model_dump()
4     elif hasattr(value, "value"):         # Python Enum
5         serializable[key] = value.value
6     else:
7         serializable[key] = value          # plain types unchanged
```

On load, `CostTracking` and `ValidationResult` are reconstructed as proper Pydantic instances. This matters because the code calls `cost_tracking.within_budget()` – that method does not exist on a plain dictionary.

9.4 The audit trail

Every time an agent runs, one record is appended to Redis:

Listing 5: AuditEntry fields

```
1 class AuditEntry(BaseModel):
2     timestamp: str          # UTC ISO timestamp
3     task_id: str
4     user_id: str
5     role: str              # the user's RBAC role
6     agent: str             # which agent ran
7     status: str            # success / error / stub
8     result_summary: str    # first 200 chars of the agent output
9     cost_usd: float        # cumulative cost at this point in the task
```

The GET /api/audit/{session_id} endpoint returns entries newest-first. Entries are never deleted, only added. The 90-day TTL is the only expiry mechanism.

10 Cost Tracking

Every LLM call costs money. The system tracks real costs using a LangChain **callback**.

A callback is an object attached to a chain or graph. When specific events occur – such as an LLM call completing – methods on the callback are triggered automatically. The `on_llm_end` method fires after every LLM call and receives the response, including token counts.

Listing 6: CostCallback accumulates cost across all LLM calls in the graph

```
1 class CostCallback(BaseCallbackHandler):
2     def on_llm_end(self, response: LLMResult, **kwargs):
3         usage = response.llm_output.get("usage", {})
4         input_tokens = usage.get("input_tokens", 0)
5         output_tokens = usage.get("output_tokens", 0)
6         cost = calculate_cost(self.model, input_tokens, output_tokens)
7         self.tracking.llm_calls += 1
8         self.tracking.tokens_used += input_tokens + output_tokens
9         self.tracking.total_cost_usd += cost
```

The callback is attached at the graph level, so it fires for every LLM call inside the graph – whether structured output, tool calling, or free text – without changing any agent code:

Listing 7: Attaching the callback when the graph runs

```
1 config = RunnableConfig(callbacks=[CostCallback(state["cost_tracking"]
2     ])])
3 final_state = await self.graph.ainvoke(state, config)
```

The pricing table in `src/utils/cost.py` covers the main Claude and OpenAI models in USD per million tokens. Calls from unknown models fall back to Claude Sonnet pricing.

11 Engineering Decisions

11.1 Supervisor pattern over dynamic planning

The original codebase used a pre-planner that generated a directed acyclic graph (DAG) of steps at runtime, plus a separate refiner to optimise it. This was replaced with a supervisor that maps intent to a fixed pipeline.

The reason: for a fintech ops team, the set of meaningful workflows is small and predictable. A dynamic planner adds complexity and failure modes without benefit. Adding a new workflow means adding one row to the routing table.

11.2 One validator with one retry

The original system had three separate critic agents (quality, security, architecture), each with veto power, and up to three retries. This was replaced with a single validator and one retry.

The reason: three critics tripled latency and cost. A single validator with a clear checklist is equally effective. One retry is enough – if two attempts both fail, the task or the data has a fundamental problem that more retries will not solve.

11.3 Structured output where decisions are made

Any agent that makes a decision or produces data that other agents depend on uses structured output. Agents that write prose use free text.

The reason: structured output failures are loud (Pydantic raises an exception immediately). Free text parsing failures are silent (the parser returns wrong data and the system continues without knowing). Loud failures are always preferable.

11.4 SQLite placeholder over a real database

An in-memory SQLite database seeded with fake data was used throughout development. All agents, tools, and tests were built against this placeholder.

The reason: this decouples development from infrastructure. The entire system can be run and tested with no external dependencies except Redis and an API key. Connecting to a real database requires changing one function: `get_db()` in `sql.py`.

11.5 Three-tier sanitization

Three separate sanitization methods exist rather than one aggressive one.

The reason: a single sanitizer that blocks all SQL keywords would reject natural language queries like “select the top funds” or “create a risk report”. The three-tier design applies the right strictness in the right context. SQL keywords are only blocked when validating an actual query string, not when reading a user’s plain-English question.

11.6 Paths anchored to the project root

Data paths are resolved relative to the source file that defines them (via `__file__`), not the current working directory.

The reason: the current working directory changes depending on how and where the process is started – which is different in a Docker container, a CI pipeline, and a developer’s terminal. Anchoring to `__file__` makes paths correct in all cases.

12 File Reference

12.1 `src/orchestrator/`

`state.py` Defines `OrchestratorState` and all the models it contains (`CostTracking`, `ValidationResult`, `Phase`, `TaskStatus`). Read this file first – it defines the shape of all data in the system.

`graph.py` The LangGraph state machine. Defines nodes, edges, the cost callback, and the `_load_agent()` dispatcher.

`coordinator.py` Accepts submissions, starts background tasks, and tracks active tasks for graceful shutdown.

12.2 `src/agents/`

`supervisor.py` Intent classification and routing. Includes `_sanitize()` as a guard against invalid LLM output.

`data.py` Tool-calling loop. Binds the three data tools. Applies RBAC table restrictions via system message.

`portfolio.py` Structured portfolio analysis. Returns a `PortfolioResult` model.

`risk.py` Structured risk assessment. Returns a `RiskResult` model with `RiskFlag` and `LimitBreach` lists.

`reports.py` Markdown report composition. Detects report type (risk, performance, mandate) from task keywords.

`validator.py` Quality gate. Returns a `ValidationResult` used by the graph’s conditional edge.

`responder.py` Final formatting pass before the response is shown to the user.

`utils.py` `extract_data_text()` – shared helper used by both portfolio and risk agents.

12.3 `src/tools/`

`sql.py` `sql.query` tool. In-memory SQLite with seeded fake data. SELECT-only with two-stage injection checks.

`excel.py` `excel.query` tool. Reads xlsx and csv. Path-safe. 50 MB and 100k row limits.

`documents.py` `document_search` tool. ChromaDB semantic search. Lazy-loaded to avoid slow startup.

`registry.py` Maps agent names to their allowed tools. The DataAgent gets all three; portfolio and risk get SQL only.

12.4 `src/gateway/`

`api.py` FastAPI application, lifespan context, and all HTTP routes. Includes `DELETE /api/task/{id}` for cancellation.

`auth.py` JWT creation and verification. `require_role()` FastAPI dependency. Role-to-table allowlist.

`sanitizer.py` Three-tier sanitization. Each method applies to a different kind of input.

`middleware.py` `slowapi` limiter with different rate limits per endpoint group.

12.5 `src/memory/` and `src/audit/`

`redis_store.py` All Redis operations: task state, session history, audit log. Handles serialisation and Pydantic reconstruction on load.

`trail.py` `AuditEntry` model and `AuditTrail` class. Fire-and-forget writes that never interrupt the main flow.

12.6 `src/utils/`

`config.py` All settings via `pydantic-settings`. Reads environment variables. Emits a warning if the API key is still the placeholder value.

`llm.py` `get_llm()` factory. Returns a `ChatAnthropic`, `ChatOpenAI`, or `AzureChatOpenAI` instance depending on the `LLM_PROVIDER` setting.

`cost.py` `CostCallback` and the pricing table. Attached to the graph via `RunnableConfig`.

`logging.py` Configures `structlog` for structured JSON output.

12.7 `scripts/` and `data/`

`scripts/ingest_docs.py` One-time ingestion. Reads `data/docs/`, chunks and embeds text, stores in ChromaDB.

`data/docs/` Two sample documents: a fund mandate and a risk policy.

`data/sample_portfolio.xlsx` Three-sheet Excel file used by `excel_query` in tests and demos.

`data/chroma/` ChromaDB vector index. Auto-generated by `ingest_docs.py`, persisted to disk.

13 What Is Not Production-Ready

This is a learning project. The architecture is sound, but the following six items need addressing before real deployment.

1. **Replace the database.** Change `get_db()` in `src/tools/sql.py` to return a connection to a real data warehouse (Snowflake, Postgres, BigQuery, etc.).
2. **Enforce authentication.** Auth is currently optional – unauthenticated requests are accepted. Add middleware that rejects requests without a valid JWT.
3. **Use a hosted vector store.** ChromaDB runs as a single local process. For multi-replica deployments, use a hosted vector database.
4. **Move secrets to a vault.** The API key and JWT secret should live in AWS Secrets Manager or HashiCorp Vault, not in a `.env` file on disk.
5. **Wire up monitoring.** A Prometheus config exists but nothing is reported. Add per-agent latency, error rate, and cost metrics.
6. **Update CORS origins.** Change `allow_origins` in `api.py` from `localhost` to the actual internal domain.

14 Where to Start Reading the Code

To understand the system by reading the source, follow this order:

1. `src/orchestrator/state.py` – the data model. Every field that can exist in a task, and its type. Understanding this file makes everything else readable.
2. `src/orchestrator/graph.py` – the workflow. Read `_build_graph()` for the structure, then each node method for the detail.
3. `src/agents/supervisor.py` – the simplest agent. Demonstrates how structured output works in practice.
4. `src/agents/data.py` – the most complex agent. Demonstrates how tool calling works.
5. `src/tools/sql.py` – a complete tool. Shows the `@tool` decorator, the security checks, and the SQLite placeholder.
6. `src/gateway/api.py` – the entry point. Shows how a request enters the system and how lifespan manages startup and shutdown.